

# Diseño y Construcción de Componentes

BISOFT-15

2C-2026

## Caso de Estudio 1

Profesor: Mario Jimenez Espinoza

Estudiante: Isayana Murillo



## Introducción

Una institución educativa desea modernizar sus sistemas de información mediante una arquitectura basada en componentes reutilizables. Actualmente, cada departamento (matrícula, cursos, estudiantes y reportes) posee aplicaciones independientes que dificultan la integración, el mantenimiento y la escalabilidad.

Para solucionar esta situación, la institución requiere el diseño de una plataforma de componentes software que permita compartir funcionalidades entre distintos sistemas, garantizando modularidad, reutilización y facilidad de integración.

El proyecto deberá enfocarse en el diseño de componentes, la publicación de interfaces y la definición adecuada de la granularidad de cada componente.

## Objetivo General

Diseñar una plataforma basada en componentes reutilizables para la gestión académica, aplicando principios de diseño de componentes, orientación a objetos, publicación de interfaces y análisis de granularidad.

## Objetivos Específicos

1. Aplicar principios de diseño de componentes para construir módulos reutilizables.
2. Implementar componentes utilizando diseño orientado a objetos.
3. Definir interfaces públicas para la comunicación entre componentes.
4. Analizar diferentes niveles de granularidad y justificar las decisiones tomadas.
5. Documentar la arquitectura propuesta y las relaciones entre componentes.

# Plataforma de Componentes para Gestión Académica Distribuida

## 1. Análisis del Problema

### Descripción de la situación

Una institución educativa requiere modernizar sus sistemas de información. Actualmente, cada departamento (matrícula, cursos, estudiantes y reportes) opera con aplicaciones independientes que generan los siguientes problemas:

- Duplicidad de información: los mismos datos se almacenan en múltiples sistemas sin sincronización.
- Dificultad de mantenimiento: un cambio en un módulo puede afectar a otros sin control.
- Escasa reutilización de código: cada sistema reimplementa funcionalidades ya existentes en otros módulos.
- Integraciones complejas: conectar los sistemas entre sí requiere esfuerzo desproporcionado.

### Solución propuesta

Se diseñó una plataforma basada en componentes de software reutilizables. Cada módulo del sistema se convierte en un componente independiente con una interfaz pública bien definida, lo que permite que distintos sistemas lo consuman sin conocer su implementación interna.

La plataforma administra cinco entidades principales: Estudiantes, Cursos, Matrículas, Docentes y Reportes académicos.

### Componentes identificados

| Componente           | Responsabilidades                                                                          |
|----------------------|--------------------------------------------------------------------------------------------|
| EstudianteComponente | Registrar, consultar, modificar y eliminar estudiantes                                     |
| CursoComponente      | Registrar, consultar, modificar y eliminar cursos                                          |
| MatrículaComponente  | Asociar estudiantes a cursos, consultar y gestionar estados: ACTIVA, CANCELADA, FINALIZADA |
| DocenteComponente    | Registrar, consultar, modificar y eliminar docentes                                        |
| ReporteComponente    | Generar reportes por estudiante, por curso y consultar estadísticas generales              |

## 2. Diagrama de Componentes

El siguiente diagrama muestra la interacción entre los componentes del sistema. Se identifican claramente el componente consumidor, los componentes proveedores, las interfaces publicadas y las dependencias hacia la base de datos.

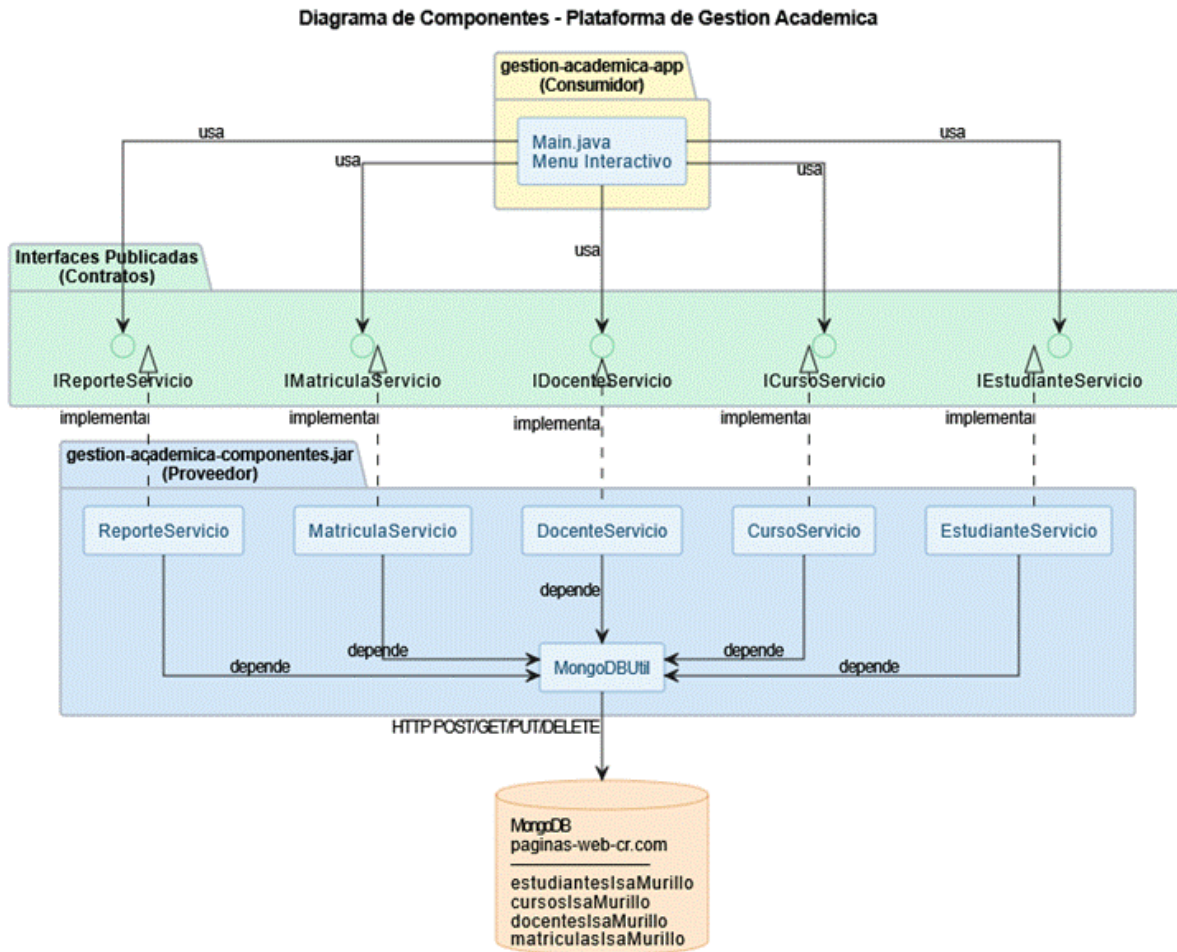


Figura 1. Diagrama de Componentes — Plataforma de Gestión Académica

### Descripción del diagrama

- Consumidor: gestion-academica-app contiene el Main.java con el menú interactivo. Es el único punto de entrada al sistema.
- Interfaces publicadas: las 5 interfaces actúan como contratos entre el consumidor y los proveedores.
- Proveedor: gestion-academica-componentes.jar contiene todos los servicios, modelos y la utilidad de conexión.
- Dependencia externa: MongoDBUtil realiza llamadas HTTP a la API de MongoDB para persistir los datos.

### 3. Diagrama de Clases

El siguiente diagrama muestra la estructura interna del componente proveedor, organizado en cuatro capas: modelo, interfaces, servicios y utilidad. Se visualizan las relaciones de herencia, composición e implementación entre las clases.

Diagrama de Clases - Plataforma de Gestion Academica

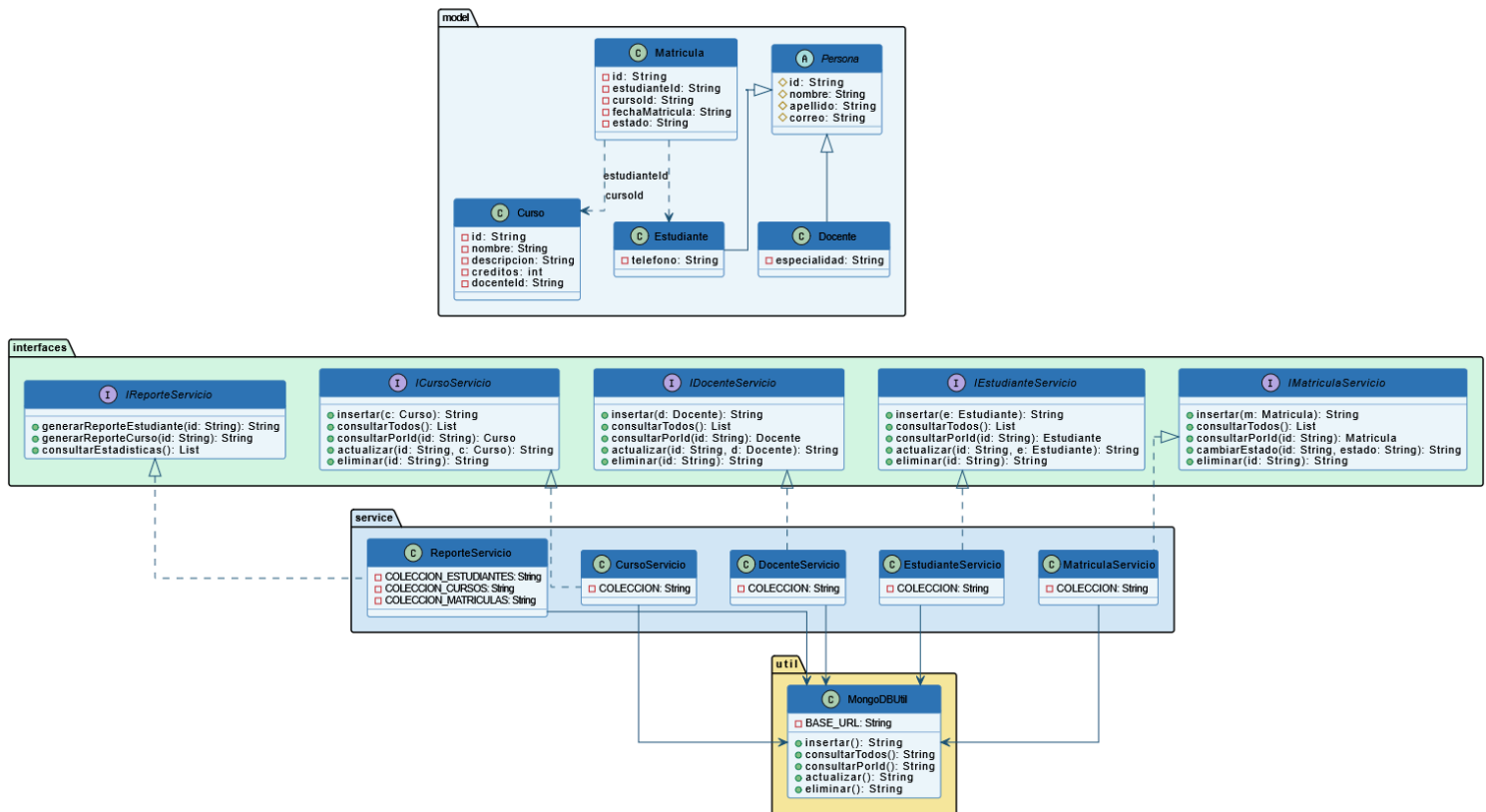


Figura 2. Diagrama de Clases — Plataforma de Gestión Académica

## Principios aplicados

| Principio       | Dónde se aplica      | Cómo                                                                             |
|-----------------|----------------------|----------------------------------------------------------------------------------|
| Encapsulamiento | Todas las clases     | Atributos private/protected, acceso solo por getters y setters                   |
| Abstracción     | Persona.java         | Clase abstract que agrupa atributos comunes y no puede instanciarse directamente |
| Herencia        | Estudiante y Docente | Ambas clases extienden Persona y heredan id, nombre, apellido y correo           |
| Composición     | Matricula.java       | Referencia a Estudiante y Curso mediante sus identificadores únicos              |

## 4. Interfaces Publicadas

Una interfaz es el contrato público que un componente le ofrece al mundo exterior. Define qué puede hacer el componente sin revelar cómo lo hace internamente. Se diseñaron cinco interfaces, una por cada componente del sistema.

| Interfaz            | Servicios que ofrece                                                 | Justificación                                                                |
|---------------------|----------------------------------------------------------------------|------------------------------------------------------------------------------|
| IEstudianteServicio | insertar, consultarTodos, consultarPorId, actualizar, eliminar       | Permite gestionar estudiantes sin conocer los detalles de MongoDB            |
| ICursoServicio      | insertar, consultarTodos, consultarPorId, actualizar, eliminar       | Desacopla la gestión de cursos del resto del sistema                         |
| IDocenteServicio    | insertar, consultarTodos, consultarPorId, actualizar, eliminar       | Permite reutilizar el componente en cualquier sistema académico              |
| IMatriculaServicio  | insertar, consultarTodos, consultarPorId, cambiarEstado, eliminar    | Expone cambiarEstado, operación exclusiva del ciclo de vida de una matrícula |
| IRaporteServicio    | generarReporteEstudiante, generarReporteCurso, consultarEstadisticas | Permite consumir reportes sin conocer cómo se generan internamente           |

### ¿Por qué favorecen la reutilización?

Las interfaces favorecen la reutilización porque separan el qué hace el componente del cómo lo hace internamente. Cualquier sistema externo que necesite gestionar estudiantes, cursos, matrículas o docentes únicamente necesita conocer la interfaz correspondiente, sin importar si la implementación interna usa MongoDB, MySQL o cualquier otra tecnología.

Si en el futuro se cambia la base de datos, los sistemas que consumen la interfaz no necesitan ningún cambio porque solo dependen del contrato, no de la implementación.

### Interfaz adicional justificada

Se agregó IRaporteServicio porque el caso de estudio menciona los reportes académicos como uno de los cinco módulos que la institución necesita administrar. Si no se publica su interfaz, ese componente queda incompleto arquitectónicamente. Con esta adición, todos los componentes exponen sus funcionalidades a través de contratos públicos bien definidos.

## 5. Análisis de Granularidad

La granularidad en componentes se refiere al tamaño y alcance de responsabilidad que tiene un componente. Se analizaron dos enfoques para el mismo problema:

### Opción A: Granularidad Fina

Cada operación es un componente separado e independiente:

- ComponenteCrearCurso — sólo se encarga de insertar un nuevo curso
- ComponenteConsultarCurso — sólo se encarga de leer cursos existentes
- ComponenteActualizarCurso — sólo se encarga de modificar un curso
- ComponenteEliminarCurso — sólo se encarga de borrar un curso

| Ventajas                                                                 | Desventajas                                                                |
|--------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Un sistema que solo necesita consultar importa únicamente ese componente | Se generan 20 componentes en lugar de 5, lo que complica la organización   |
| Se puede modificar una operación sin tocar las demás                     | Coordinar dos componentes distintos para operaciones relacionadas          |
| Permite restringir permisos de forma muy específica por operación        | Mayor complejidad al integrar sistemas que necesitan múltiples operaciones |

### Opción B: Granularidad Gruesa

Todas las operaciones CRUD se agrupan en un único componente por entidad:

- ComponenteCurso — incluye insertar, consultar, actualizar y eliminar en una sola clase

| Ventajas                                                           | Desventajas                                                                |
|--------------------------------------------------------------------|----------------------------------------------------------------------------|
| Un solo componente agrupa toda la lógica de una entidad            | Un sistema que solo consulta igualmente debe cargar el componente completo |
| Solo 5 componentes que gestionar en lugar de 20                    | Un cambio requiere recompilar y redistribuir todo el componente            |
| El componente es cohesivo: todo lo de Cursos está en un solo lugar | Menor flexibilidad para sistemas que requieren solo una operación          |

Decisión: Granularidad Gruesa

Se utilizó la Opción B: Granularidad Gruesa. El sistema académico requiere que sus componentes sean cohesivos y fáciles de integrar. En este contexto, las operaciones sobre una misma entidad siempre se usan en conjunto. Cualquier pantalla que gestione cursos necesitará tanto consultar como insertar, actualizar y eliminar.

Esta decisión se refleja directamente en la estructura del proyecto: en lugar de 20 clases de servicio, el sistema tiene únicamente 5 servicios cohesivos, uno por cada entidad del dominio académico.

## 6. Conclusiones y Recomendaciones

### Conclusiones

La arquitectura basada en componentes permite que cada módulo del sistema sea desarrollado, mantenido y distribuido de forma independiente, reduciendo el acoplamiento entre sistemas.

La aplicación de los principios de orientación a objetos resultó en un diseño limpio donde cada clase tiene una responsabilidad clara: los modelos representan datos, las interfaces publican contratos y los servicios implementan la lógica de negocio.

Las interfaces como contratos públicos garantizan que los sistemas consumidores no dependan de los detalles internos de implementación, lo que facilita futuros cambios tecnológicos sin afectar a quien consume el componente.

La granularidad gruesa fue la decisión correcta para este contexto académico, ya que las operaciones sobre una misma entidad siempre se usan en conjunto, y mantenerlas en un único componente reduce la complejidad de integración.

El uso de MongoDB como base de datos en la nube a través de una API REST demostró ser una solución práctica y moderna que permite persistir datos sin necesidad de instalar ni configurar una base de datos localmente.

La separación en dos proyectos Maven demuestra en la práctica el concepto de reutilización: el archivo `.jar` del componente puede ser usado por cualquier otro proyecto sin modificación alguna.

## **Recomendaciones**

Se podría agregar un componente de Seguridad que maneje autenticación y roles de usuario, completando así la arquitectura con control de acceso. Se recomienda explorar el manejo de excepciones personalizado en cada servicio, para que los errores de la API se comuniquen al usuario con mensajes claros en lugar de mensajes técnicos. Para continuar profundizando en el tema, sería valioso investigar cómo funcionan los pipelines de integración continua (CI/CD), que permiten compilar, probar y distribuir automáticamente los componentes cada vez que se realiza un cambio en el código.